

Kubernetes の HPA 環境下におけるコンテナ集約の適用

田尾瑞紀

複数サービスを 1 つの Pod に同居させる**部分集約**は通信コストを下げるが、Kubernetes では HPA が Deployment 単位で台数を決めるため、集約すると**スケールの粒度が粗くなり**予約資源が増える。本稿では Online Boutique を用いた実機実験により、集約がスループットと通信コストを改善する一方、この損失の大きさが**通信量とワークロード成分**という独立な 2 軸で決まることを示す。

1 はじめに

マイクロサービスを細分化すると、サービス間の呼び出しがネットワークスタックを経由するため、通信処理のオーバーヘッドが生じる。直接通信するサービスを 1 つの Pod に同居させ、通信を localhost 経由にする**部分集約**はこれを削減する。Wickramanayaka ら [2] は Docker Swarm 環境で、通信量の多いコンテナの適正配置・統合を行い通信量が最大 58.5% 減少することを示した。

この効果を、実運用のデファクトスタンダードである Kubernetes に取り入れたい。しかし中核機能である HPA は Deployment 単位で台数を決めるため、集約すると**スケールの粒度が粗くなる**という問題が生じる。本稿ではこの環境での効果を確認したうえで、相手選びに必要な判断軸を示す。

2 部分集約とオートスケーラの干渉

HPA は Deployment 単位で台数を決めるため、同居したコンテナは**同じ台数でしか複製できず**、その台数は各コンテナが必要とする台数の最大値になる。サービス s の枠 (requests) を $r(s)$ 、単独に必要な台

表 1 2 種類の利用者が到達するサービス

利用者	到達するサービス
buy	全 10 サービス
view	frontend, productcatalog, cart, currency, recommendation, ad

数を $n(s)$ 、同居した Pod の台数を $N = \max_s n(s)$ とすると、余分に確保される資源 Δ は

$$\Delta = \sum (N - n(s)) \times r(s) \quad (1)$$

となる。損は「**引きずられた側の枠**」に比例し、必要台数が揃えば $\Delta = 0$ である。以降この Δ を**粒度損**と呼ぶ。

3 実験環境

対象は Online Boutique [1] (11 サービス, gRPC 通信) である。MicroK8s (単一ノード, 16 コア) 上にデプロイし, k6 を用いて負荷を与えた。

負荷は 2 種類の利用者を模した (表 1)。**buy** は閲覧・カート追加・購入を、**view** は閲覧のみを 1 周とする。**view は checkout・payment・shipping・email に一切到達しない**。以降, 1 秒あたり x 周のレートで与える負荷を $\text{buy}(x) \cdot \text{view}(x)$ と表す。

評価指標は、粒度損として**予約枠** (稼働 Pod の requests 合計)、通信コストとして **softirq の CPU 時間**を用いた。

3.1 予備実験: Kubernetes 環境での効果

分離 (1 Pod 1 サービス), frontend・recommendation・productcatalog を同居させた **front3**, 全サービスを同居させた **mega** の 3 構成を比較した. 台数は 4 で固定して HPA を切り, 全コンテナの枠を 150m に揃えたので, 予約資源はどの構成も 6.3 コアで一致する (3 回反復, buy のみ).

buy(150)–buy(300) ではいずれも目標レートを達成し, 取りこぼしと失敗は 0 であった. その条件で 1 周あたりの softirq は分離に対し front3 で 17–23%, mega で 43–51% 少なく, **同居させるほど通信コストが下がる**. 応答時間は buy(300) で分離だけが不安定になり (p50 が 23–79 ms, 同居は 16–21 ms), 過負荷 (buy(500)) では達成レートが分離 348 周/秒に対し front3 425, mega 435 と, **同じ予約資源でより多くを処理できた**.

4 本実験

以降の実験では, 枠 (requests) はマニフェストの既定値ではなく, 実測から適正化した値を用いる. 各サービスを 1 台に固定し HPA を切った状態で負荷を 6 段階 (10–150 周/秒) 与え, CPU 使用量を負荷の 1 次式として最小二乗法で近似し,

$$r = (c_0 + c_1 k) / 0.7, \quad k = 100 \quad (2)$$

とした. ここで c_0 はアイドル時の CPU 使用量, c_1 は 1 リクエストあたりの CPU 時間, k は基準負荷 [周/秒] である. これにより, 理想的には全サービスが同じ負荷で目標利用率に到達し, 必要台数が一致する. その上で, buy(100) を一定に保ちつつ view を 4 段階 (0・100・200・250 周/秒) で変える実験を行った.

4.1 ワークロード成分が異なると損は消せない

buy のみを与えた場合にも粒度損は生じたが, これは email の利用率が増設ラインをわずかに超えた (83%) ため, **枠が 7%不足していただけであり**, buy(150) では両者とも 2 台となって消える (表 2 の上 2 行). すなわち**枠の測定精度に起因する損は除去できる**.

これに対し view(0) を view(300) にすると, **front**

表 2 条件による粒度損の変化 (FE+email, いずれも 3/3 で一致)

buy	view	必要台数	引きずら	粒度損
[周/秒]		FE/email	れる側	[mCPU]
100	0	1 / 2	FE (1077 m)	+1077
150	0	2 / 2	—	0
100	300	3 / 2	email (119 m)	+119

tend の需要は 2.5 倍になるのに対し email は 1.2 倍にしかならない (表 2 の 1 行目と 3 行目).

同じ組み合わせでありながら, 引きずる側と引きずられる側が入れ替わり, 損が 9 倍変化した. 損の比 $1077/119 = 9.05$ は枠の比と一致し, 式 (1) が両方向で成立している.

枠は各サービスに 1 つしか与えられないのに対し, 需要の比はワークロードの混合比によって変化するため, **2 つの条件で同時に損をゼロにする枠は存在しない**. すなわちこの損は枠の調整では原理的に除去できない.

ただし成分が異なれば必ず損が出るわけではなく, 必要台数が一致した際の粒度損はゼロであった.

5 考察

便益は通信量で決まり, 枠を 3 倍変えても softirq の削減率は 16.4%・16.2% とほぼ変わらない. 一方, 損は成分で決まる. この独立な 2 軸から, 避けるべきは「**通信量が多く, かつ成分が異なる**」組み合わせとなる. しかし実測では太い通信の両端はいずれも同一成分で, 成分をまたぐ通信は最大 2.6% であり, 本アプリケーションに**この組み合わせは存在しなかった**. 便益の大きい相手を選ぶ限り, 粒度損の危険は踏まないことになる.

6 今後の課題

成分を跨ぐアプリケーションでの実行.

また本実験は単一ノードで行っており, ノードをまたぐ通信 (overlay ネットワーク) を含まない. 先行研究 [2] が示した効果の大部分は overlay の有無に由来するため, マルチノード環境では便益がさらに大きくなると予想される.

参考文献

- [1] Google Cloud Platform: Online Boutique, GitHub リポジトリ `GoogleCloudPlatform/microservices-demo`, 2025.
- [2] Wickramanayaka, I., Keppitiyagama, C., and Thilakarathna, K.: Communication-Affinity Aware Colocation and Merging of Containers, *International Journal of Advances in ICT for Emerging Regions*, Vol. 15, No. 3(2022).